

Dynamic Expert Allocation for Code Summarization: A Mixture-of-Experts Approach and Its Limitations

Hammad Saleem

GCUF, Department of Computer Science | hammadswab@gmail.com

1. Abstract

Code summarization models are typically trained as a single dense model that treats all code structures uniformly, despite meaningful structural differences between standalone functions and class methods. This paper investigates whether routing code to structurally-specialized expert models, via a lightweight gating mechanism inspired by Mixture-of-Experts (MoE) architectures, improves summarization quality over a single dense baseline. Using a fine-tuned CodeT5+ (220M) baseline and two category-specialized experts (method vs. standalone function, determined by AST-based parsing) evaluated on a held-out CodeSearchNet Python subset, we find that the full MoE system does not outperform the dense baseline: BLEU degrades from 0.133 to 0.073, ROUGE-L from 0.274 to 0.260, and per-example latency increases roughly 4-fold. Component-level analysis attributes this to class imbalance in the training data: the majority category (method, 56% of examples) is already well-served by the dense baseline's data volume, so isolating it into a smaller dedicated expert removes useful data without a compensating specialization benefit, while the minority category (function, 44%) shows a small genuine improvement from specialization. We also report a significant methodological finding: an initial baseline exhibited near-perfect BLEU (0.999) due to unintentional docstring leakage in the CodeSearchNet raw code field, which was diagnosed and corrected prior to all reported results. We conclude that structural routing alone is not sufficient to guarantee MoE gains, and that per-expert data volume is a confounding variable that must be controlled for or explicitly modeled.

Keywords: Mixture of Experts, Code Summarization, CodeT5, Gating Networks, BLEU, ROUGE, Class Imbalance.

2. Introduction

2.1 Problem

Automatic code summarization — generating a natural-language description of a code snippet's purpose — is a well-studied task supporting documentation generation, code search, and developer onboarding. Standard approaches fine-tune a single sequence-to-sequence model (e.g., CodeT5) on a mixed corpus of functions, methods, and scripts, treating structurally distinct code the same way despite differing context dependency and documentation conventions.

2.2 Motivation

Mixture-of-Experts (MoE) architectures address a structurally similar problem in large language models by routing inputs to specialized sub-networks rather than forcing one dense network to handle all input types. This project asks whether the same principle, applied at a coarse, code-structural granularity, yields a measurable improvement for code summarization specifically — and, if not, why not.

2.3 Research Question

Does routing code to a category-specialized summarization model (method-expert vs. function-expert), selected by a lightweight gating function, produce better summaries than a single dense model trained on the same combined data — and at what latency cost?

2.4 Hypothesis

H1: A Mixture-of-Experts system, where each expert is fine-tuned only on its structural category, will achieve higher BLEU/ROUGE than a single dense model trained on all categories combined, at some added latency cost from routing. This hypothesis is treated as falsifiable throughout; Sections 8–9 report and analyze the outcome regardless of direction.

2.5 Contributions

- A working end-to-end MoE pipeline for code summarization: AST-based structural categorization, category-specialized fine-tuned experts, a rule-based gate, and full system evaluation against a dense baseline.
- Identification and correction of a docstring-leakage confound in CodeSearchNet, which silently inflated an initial baseline's BLEU score to 0.999 by exposing the reference summary inside the model's input.
- A component-level analysis isolating class imbalance as the primary driver of the MoE system's underperformance, rather than treating the result as an undifferentiated negative outcome.
- An explicit accounting of a latency measurement confound (batched baseline generation vs. unbatched MoE generation) rather than reporting the raw comparison uncritically.

3. Background

3.1 Code Summarization

Code summarization casts documentation generation as a sequence-to-sequence task: source code as input, a natural-language description as output. Model quality is conventionally measured by overlap with a human-written reference summary, most commonly via BLEU or ROUGE (Section 3.4).

3.2 Transformers / CodeT5+

CodeT5 and its successor CodeT5+ are encoder-decoder Transformer models pretrained on code-specific corpora, widely used as a base for fine-tuning on code summarization and related tasks. This work uses CodeT5+ (220M parameters) as the shared architecture for both the dense baseline and all specialized experts, isolating training-data partitioning as the only independent variable of interest.

3.3 Mixture of Experts

A Mixture-of-Experts system consists of multiple specialized sub-models (experts) and a gating function that routes each input to one or more experts. MoE is most commonly used to scale model capacity efficiently in large language models, where routing is learned jointly with the experts. This study instead evaluates a coarser, structurally-motivated routing scheme with a small, fixed number of experts and a rule-based gate, representative of how MoE-style specialization might be applied in a smaller-scale, domain-specific deployment.

3.4 BLEU and ROUGE

BLEU and ROUGE measure n-gram overlap between generated and reference text. Both are known to penalize semantically correct but differently-phrased output, a limitation directly observed in this study's qualitative analysis (Section 9) and worth foregrounding before interpreting the numerical results in Section 8.

4. Related Work

Sparse Mixture-of-Experts layers were popularized in large-scale language modeling as a means of increasing model capacity without proportionally increasing inference cost, with routing learned end-to-end alongside the experts [4]. Code-specific pretrained models such as CodeT5 and CodeT5+ [1, 2] have become standard bases for fine-tuning on code summarization, typically as a single dense model over a mixed-category corpus. CodeSearchNet [3] is the standard large-scale corpus underlying much of this work, pairing functions with natural-language docstrings extracted from open-source repositories; this study also surfaces a specific data-quality artifact in that corpus (Section 5.3) relevant to any work training directly on its raw code field. Unlike learned, fine-grained MoE routing in large-scale LLMs, this work evaluates coarse, structurally-defined, rule-based routing at a much smaller model and data scale, closer to how a resource-constrained team might realistically apply the MoE principle.

5. Dataset and Preprocessing

5.1 CodeSearchNet

Experiments use the Python subset of CodeSearchNet, sub-sampled to 3,000 training and 300 test examples for computational tractability on a single free-tier GPU (Colab T4). Each example pairs a function's source code with its human-written docstring, extracted from real open-source repositories.

5.2 Data Selection

A fixed 3,000/300 train/test sub-sample was used throughout, sized to permit full fine-tuning of every model configuration (baseline, two experts) within Colab's free-tier session limits, while remaining large enough to produce a non-trivial fine-tuning signal.

5.3 Docstring Leakage Removal

An initial baseline model, trained directly on CodeSearchNet's raw `func_code_string` field, achieved a near-perfect BLEU score of 0.999 and ROUGE-L of 0.999 after three fine-tuning epochs. Rather than accepting this result, qualitative inspection of model outputs was performed, which revealed the model was reproducing its input verbatim rather than generating a genuine summary.

Root-cause analysis identified that CodeSearchNet's raw code field includes the function's own docstring as part of its source text — the same text used as the training target. The model therefore had no incentive to learn genuine summarization; copying the first quoted text block in its input was sufficient to minimize training loss.

The correction applied an AST-based transformation that parses each function, locates and removes its docstring node from the function body prior to tokenization, and re-serializes the resulting code with `ast.unparse`. All results reported in this paper (Sections 8–9) use this corrected input; the leaky baseline is reported here only as a methodological finding, illustrating why the discovery deserved a dedicated subsection rather than a passing mention: it changed the validity of every downstream result, not merely one number.

5.4 Code Categorization

An initial categorization scheme (standalone function / class / script), based on the presence of `ast.FunctionDef` and `ast.ClassDef` nodes, was applied first. This revealed that CodeSearchNet's Python subset consists almost exclusively of isolated single-function snippets (2,975 of 3,000 examples classified as “function”; 25 as “script”; 0 as “class”), since each example is extracted as a standalone function rather than a full source file. This scheme was therefore abandoned as unusable for defining balanced experts.

A revised scheme classifies each example as method or function based on whether its first parameter is `self` or `cls` (indicating it was originally a class method, even though the surrounding class definition is not present in the isolated snippet), versus a genuinely standalone function. This produced a usable, approximately balanced split (Table I).

TABLE I. Category distribution after AST-based method/function classification.

Split	Method	Function	Total
Train	1,677	1,298	2,975
Test	175	125	300

6. Methodology

6.1 Dense Baseline

A single CodeT5+ model fine-tuned on the full, leakage-corrected training set (2,975 examples, both categories combined). Serves as the point of comparison for every other configuration.

6.2 Specialized Experts

Two independently fine-tuned copies of the same base model: Expert-Method, trained only on method-labeled examples (1,677); Expert-Function, trained only on function-labeled examples (1,298). Identical training configuration to the baseline, differing only in training-data composition.

6.3 Routing / Gate

A rule-based gate reusing the AST method/function classifier directly, guaranteeing 100% routing accuracy on this dataset by construction. This choice isolates the effect of expert specialization from gate error, at the cost of not testing routing robustness under imperfect classification (see Section 10).

6.4 Complete MoE Pipeline

Raw code → gate classifies as method or function → input is routed to the corresponding fine-tuned expert → the selected expert generates the summary (greedy decoding, capped at 64 tokens). The dense baseline bypasses gating and always uses the single shared model.

7. Experimental Setup

7.1 Hardware

All training and evaluation was performed on a single Google Colab free-tier NVIDIA T4 GPU (16GB VRAM), using fp16 mixed-precision training throughout.

7.2 Training Configuration

- Base model: Salesforce/codet5p-220m, fine-tuned independently for each of the three configurations (baseline, Expert-Method, Expert-Function).
- Epochs: 3, with evaluation and checkpointing after each epoch.
- Batch size: 8 (train and eval); Learning rate: 5e-5; Precision: fp16.
- Max input length: 256 tokens (code); max target length: 64 tokens (summary).

7.3 Evaluation Metrics

BLEU and ROUGE-1/2/L/Lsum (via the HuggingFace evaluate library), computed on generated vs. reference summaries for each configuration's respective test set. Per-example latency (wall-clock seconds) and gate routing accuracy are additionally reported for the full MoE pipeline (Section 8.4).

8. Results

8.1 Baseline Results

Trained on the full, leakage-corrected training set and evaluated on the full 300-example test set:

TABLE II. Dense baseline results (docstring leakage corrected).

Metric	Score
BLEU	0.133
ROUGE-1	0.315
ROUGE-2	0.120
ROUGE-L	0.274
ROUGE-Lsum	0.301

8.2 Expert Results

Each expert is compared against the dense baseline restricted to the same category's test subset, isolating the effect of specialization from the effect of dataset composition.

TABLE III. Baseline vs. specialized expert, evaluated per-category.

Test Category	Model	BLEU	ROUGE-L
Method (n=175)	Dense Baseline	0.149	0.297
Method (n=175)	Expert-Method	0.132	0.275
Function (n=125)	Dense Baseline	0.082	0.242
Function (n=125)	Expert-Function	0.088	0.248

On the majority category (method), the dense baseline outperforms the specialized expert. On the minority category (function), the specialized expert shows a small improvement over the baseline. This asymmetry is analyzed in Section 9.

8.3 MoE Results

The complete pipeline (gate + both experts) is evaluated on the full 300-example test set, directly comparable to Table II.

TABLE IV. Full system comparison: dense baseline vs. complete MoE pipeline.

Metric	Dense Baseline	Full MoE System	Δ
BLEU	0.133	0.073	−0.060
ROUGE-1	0.315	0.299	−0.016
ROUGE-2	0.120	0.112	−0.008
ROUGE-L	0.274	0.260	−0.014
Routing Accuracy	—	100%	—

8.4 Latency

TABLE V. Per-example latency: dense baseline vs. full MoE pipeline.

Configuration	Latency / example (s)
Dense Baseline (batched, size 8)	0.222
Full MoE System (unbatched)	0.857

The full MoE system underperforms the dense baseline on every quality metric, and is approximately 3.9× slower per example. This comparison is confounded by an implementation difference addressed directly in Section 9.3.

9. Discussion

9.1 Why MoE Did Not Beat Baseline

The overall MoE degradation is explained by the component-level results in Table III rather than a uniform failure of specialization. Method examples constitute 56% of the training data; the dense baseline, despite serving both categories, therefore already receives substantial method-specific signal purely through data volume — effectively specializing on the majority class implicitly. Isolating method examples into a dedicated, smaller expert removes the (comparatively small) cross-category signal the dense model was also benefiting from, without adding new information, so the dedicated expert underperforms. Because the test set is majority-method, this effect dominates the blended system's overall score.

9.2 Effect of Data Distribution

On the minority class (function, 44% of data), the dense baseline's attention is comparatively diluted, so a dedicated expert — despite having strictly less absolute training data — achieves a small net improvement. This indicates MoE-style specialization benefits are conditional on relative class imbalance and per-expert data volume, not a guaranteed consequence of structural category separation. A dataset with more balanced categories, or with experts trained on proportionally larger data, may show a different outcome; this is proposed directly as future work (Section 11).

9.3 What Worked

- The AST-based docstring-leakage fix produced a plausible, non-degenerate baseline (BLEU 0.133) consistent with published ranges for small-scale code summarization fine-tuning.
- The rule-based gate achieved perfect routing accuracy by construction, confirming the wiring of the full pipeline (categorization → routing → expert selection → generation) is correct.
- Specialization produced a small, genuine improvement on the minority category (function: BLEU 0.082 → 0.088), showing the MoE principle is not universally invalid — only insufficient on its own at this data scale and imbalance.

9.4 What Did Not Work

- Overall system quality: the full MoE pipeline underperformed the dense baseline on every aggregate metric (Table IV).
- Latency: the MoE pipeline was measured at roughly 3.9× the baseline's per-example latency, though this comparison is confounded — the baseline used batched generation (size 8) while the MoE loop generated one example at a time as routing decisions were made per-example; a category-batched MoE implementation would likely close much of this gap and was not implemented here.
- Reference-label quality: qualitative inspection found CodeSearchNet reference summaries are occasionally stray provenance comments rather than functional descriptions, undermining automatic metrics for both configurations equally but not correctable within this study's scope.

Representative qualitative examples, illustrating both metric-penalized correct paraphrasing and reference-label noise:

- Predicted: “Get Youtube VID from a url” — Reference: “Extracts video ID from URL.” (semantically equivalent, penalized by n-gram overlap)
- Predicted: “Create a new mimi hash” — Reference: “From <http://cdn37.atwikiimg.com/>... makeMimiLocal L110” (prediction arguably more accurate than a reference that is a provenance comment, not a functional description)
- For longer, more complex functions (e.g., video-download wrappers), the model generated generic docstring-style templates (“Args: url (str)... Returns:...”) not grounded in the function's specific logic — evidence the ~3,000-example training set was insufficient for deep per-function specificity on more complex code.

10. Limitations

- Small dataset scale (2,975 train / 300 test) relative to typical code summarization benchmarks; results may not generalize to larger-scale training.
- Gate is rule-based (reuses the ground-truth AST classifier), yielding 100% routing accuracy by construction; a learned gate operating on raw text without AST access would likely show non-trivial routing error, which was not evaluated here.

- Latency comparison is confounded by batched vs. unbatched generation (Section 9.3).
- Only two categories (method, function) were evaluated; a finer-grained or differently-defined taxonomy (e.g., by cyclomatic complexity) was not tested.
- BLEU/ROUGE were the sole automatic metrics used; no human or LLM-judge evaluation was performed, despite qualitative evidence that these metrics understate true output quality in several cases.

11. Future Work

- Repeat the experiment with proportionally scaled training data per expert, to isolate specialization benefit from the data-volume confound identified in Section 9.2.
- Implement batched, category-grouped MoE generation to obtain an unconfounded latency comparison.
- Evaluate a learned (non-AST) gating classifier and report its routing accuracy and its downstream effect on quality when imperfect.
- Incorporate a human or LLM-judge evaluation pass to address the demonstrated limitations of BLEU/ROUGE on this task.
- Extend to a 3-way or complexity-based categorization scheme, and to additional programming languages.

12. Conclusion

This study set out to test whether category-specialized Mixture-of-Experts routing improves code summarization over a single dense model. The hypothesis was not confirmed: the full MoE system underperformed the dense baseline on every quality metric and showed substantially higher (though confounded) latency. Component-level analysis attributes this to class imbalance — specialization helped the minority category and hurt the majority category, and the net effect on a majority-skewed test set was negative. A significant secondary finding — docstring leakage silently producing a near-perfect but meaningless initial baseline — underscores the necessity of qualitative output inspection alongside aggregate metrics in any code-summarization pipeline built on CodeSearchNet-derived data. For small-scale, single-language tasks with imbalanced structural categories, a single well-trained dense model is likely preferable to explicit MoE routing unless per-expert training data can be scaled to match what each category would receive under a combined dense model.

13. References

- [1] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, “CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation,” EMNLP, 2021.
- [2] Y. Wang et al., “CodeT5+: Open Code Large Language Models for Code Understanding and Generation,” arXiv:2305.07922, 2023.
- [3] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, “CodeSearchNet Challenge: Evaluating the State of Semantic Code Search,” arXiv:1909.09436, 2019.
- [4] N. Shazeer et al., “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,” ICLR, 2017.

[5] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a Method for Automatic Evaluation of Machine Translation," ACL, 2002.

[6] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," ACL Workshop, 2004.